

用語集 モノイド 群 環 体 有限体 多項式・形式的べき級数 行列 畳み込み

代数構造用語集

1. 概要

本講座では、競技プログラミング（あるいは AtCoder Algorithm Lectures）での使用頻度が高い、代数構造に関する用語および、その代表例について解説します。

代数構造は典型的には、集合と集合上の演算の組として定義されます。例えば、整数に対する演算として加法、乗法、 $\min, \max$, $\gcd$ などを考えることが、代数構造を考えることに対応します。代数構造は、単位元や逆元はあるのかなどといった性質によってある程度分類されており、本講座ではそのような分類を整理しています。

このような考え方は、特に定理やアルゴリズムを**抽象化・一般化**する際に役に立ちます。例えばセグメント木の理解の方法のひとつに、**モノイド**の区間積クエリが扱えるというものがあります。このような抽象的な理解の方法に慣れると、セグメント木を加法、乗法、 $\min, \max$, $\gcd$ などに利用できることが個別の演算について詳しく考えずともスムーズに理解できるようになります。またプログラミングでの実装においても、同じアルゴリズムをより多くの状況で使えるように実装するといった工夫ができるようになります。

本講座の内容は抽象度が高く、通常大学レベルの数学で扱われる内容です。高校までの数学ではあまり馴染みのないものが多いため、初めはなかなか慣れられず、このような考え方に利点を感じられないかもしれません。しかし、競技プログラミングの学習を続けていくと、必然的にこれらの代数構造の具体例に多く触れることになります。最初は本講座の内容が上手く理解できなかった場合にも、AtCoder Algorithm Lectures やその他の競技プログラミングの学習を通して様々な代数構造の例に触れながら、ときどき本講座を見返していただければ、徐々にこれらの概念への抵抗がなくなっていくと思います。

2. 前提知識

集合・写像の用語および、合同式 ($\mathbb{Z}/n\mathbb{Z}$), 多項式などの例を知っていることを前提とします.

それ以上の前提知識がなくとも概念理解は可能です. ただし, 代数構造が競技プログラミングにおいてどのように現れるかを説明している部分では, 競技プログラミングの話題を広く知っていることを前提に説明している場合があります. そのような箇所は, 初習の際には分からない内容はある程度読み飛ばしていただくのが良いと思います.

3. 二項演算

3.1. 二項演算とは

【定義 1】

S を集合とする. S の要素の 2 個組 (a, b) に対して S の要素 $a * b \in S$ を定める規則 $*$ を S 上の**二項演算** (binary operation) という. 言い換えれば, 二項演算とは直積集合 $S \times S$ から S への写像である.

例えば $S = \mathbb{Z}$ (整数全体の集合) とするとき, 次は二項演算です.

1. $a * b = a + b$.
2. $a * b = a - b$.
3. $a * b = a \times b$.
4. $a * b = \max(a, b)$.
5. $a * b = \min(a, b)$.
6. $a * b = \gcd(a, b)$.
7. $a * b = a$.
8. $a * b = 1$.

除算 $a * b = a/b$ は, $b = 0$ の場合に定義されないことや, 商が整数ではない有理数となることがあるため, $\mathbb{Z}$ 上の二項演算では**ありません**.

上の例 4, 5, 6 のように, 二項演算は引数を 2 つ持つ関数への代入として表記されることがあります.

3.2. 結合法則・交換法則

【定義 2】

S を集合, $*$ を S 上の二項演算とする. 任意の $a, b, c \in S$ に対して

$$(a * b) * c = a * (b * c)$$

が成り立つとき, $*$ は**結合法則** (associative law) を満たすという.

結合法則は二項演算の性質のうちでも特に基礎的なもので, 本講座に登場する二項演算でもほぼ常に結合法則を仮定することになります.

結合法則が直接述べているのは, 「二項演算の反復で 3 つの要素を 1 つの要素にすると, 結果が操作手順に寄らない」ということですが, これが成り立つ場合に, より一般に「二項演算の反復で n 個の要素を 1 つの要素にすると, 結果が操作手順に寄らない」ことも証明可能です. 特に $*$ が結合的であるとき, S の要素 $a_1, a_2, \dots, a_n$ に対して, 括弧をつけずに $a_1 * a_2 * \dots * a_n$ という表記をすることがよくあります. ここでは一般的な証明を記述することはありませんが, ひとつ例を挙げておきます.

【問題 3】

S 上の二項演算 $*$ が結合法則を満たすとき, $(a * b) * (c * (d * e)) = (((a * b) * c) * d) * e$ を証明してください.

▶ 解答例

【定義 4】

S を集合, $*$ を S 上の二項演算とする. $a, b \in S$ に対して

$$a * b = b * a$$

が成り立つとき, $*$ は**交換法則** (commutative law) を満たすという.

交換法則も結合法則とともに二項演算に対する代表的な要請ですが、交換法則を満たさない二項演算を考えることも多いです。

3.3. 単位元・逆元

【定義 5】（単位元）

S を集合， $*$ を S 上の二項演算とする． $e \in S$ が**単位元**（identity element）であるとは，任意の $a \in S$ に対して

$$e * a = a * e = a$$

が成り立つことをいう．

【命題 6】

S 上の二項演算 $*$ の単位元は，存在するならば一意である．

e_1, e_2 がともに単位元であるとする， $e_1 = e_1 * e_2 = e_2$ より $e_1 = e_2$ となるので示された． ■

【定義 7】（逆元）

$*$ を S 上の二項演算とし， $e \in S$ がその単位元であるとする． $a, b \in S$ について

$$a * b = b * a = e$$

が成り立つとき， b は a の $*$ に関する**逆元**（inverse element）であるという．

【命題 8】

S 上の二項演算 $*$ が結合法則を満たし，単位元をもつとする．このとき $a \in S$ の $*$ に関する逆元は，存在するならば一意である．

b_1, b_2 が逆元であるとする、

$$b_1 = b_1 * e = b_1 * a * b_2 = e * b_2 = b_2$$

となるのでよい. ■

4. モノイド

4.1. 定義

【定義 9】

集合 M と M 上の結合的な二項演算 $*$ の組 $(M, *)$ を**半群** (semigroup) という. さらに $*$ の単位元が存在するとき**モノイド** (monoid) という.

半群・モノイドにおいてその二項演算が交換法則を満たすとき, **可換半群** (commutative semigroup)・**可換モノイド** (commutative monoid) という.

モノイドをはじめとして, 本講座内で以降定義する群・環・体といった代数構造は, このように「集合と演算の組」として定義されます. ただし, 文脈上ひとつの二項演算を固定して考えていることが明らかな場合などに, 集合の記号と区別せずに「 M をモノイドとする」といった表現を用いることもよくあります. あるいは「 M をモノイド, $*$ をその二項演算とする」といった表現を用いることもあります.

半群は, 単位元に相当する特別な要素を付け加えることでモノイドになります.

【命題 10】

$(S, *)$ を半群とする. このとき, 新たに特別な要素 e を考え, $S' = S \cup \{e\}$ 上に二項演算 $*'$ を

$$a *' b = \begin{cases} a * b & (a, b \in S) \\ b & (a = e, b \in S) \\ a & (a \in S, b = e) \\ e & (a = b = e) \end{cases}$$

により定めると、 $(S', *)$ はモノイドになる。

結合法則を確かめればよく、簡単な場合分けにより示されるので省略します。 ■

このこともあり、現在の日本の競技プログラミングコミュニティでは、半群に関する議論はほとんど行われておらず、モノイドを扱うことが主流となっています。

4.2. 具体例

モノイドの例のうち基本的なものをいくつか挙げます。

S	二項演算	単位元	交換法則
$\mathbb{Z}$ (整数全体)	$+$	0	○
$\mathbb{Z}$	$\times$	1	○
$\{0, 1, \dots, n-1\}$ ($\mathbb{Z}/n\mathbb{Z}$)	$(a+b) \bmod n$	$0 \bmod n$	○
$\{0, 1, \dots, n-1\}$ ($\mathbb{Z}/n\mathbb{Z}$)	$(ab) \bmod n$	$1 \bmod n$	○
$\mathbb{Z} \cup \{\pm\infty\}$	$\min$	∞	○
$\mathbb{Z} \cup \{\pm\infty\}$	$\max$	$-\infty$	○
$\mathbb{Z}$	$\gcd$	0	○
$\mathbb{Z}_{\geq 0}$ (非負整数全体)	XOR	0	○
$\mathbb{Z}_{\geq 0}$	OR	0	○
文字列全体	文字列の結合	空文字列	×
写像 $\mathbb{Z} \rightarrow \mathbb{Z}$ 全体	写像の合成	恒等写像	×
$x \mapsto ax + b$ ($a, b \in \mathbb{Z}$) と書ける写像 $\mathbb{Z} \rightarrow \mathbb{Z}$ 全体	写像の合成	恒等写像	×
$x \mapsto \min\{x + a, b\}$ ($a \in \mathbb{Z}, b \in \mathbb{Z} \cup \{\infty\}$) と書ける写像 $\mathbb{Z} \rightarrow \mathbb{Z}$ 全体	写像の合成	恒等写像	×

S	二項演算	単位元	交換法則
整数成分の $n \times n$ 行列全体 $M_n(\mathbb{Z})$	行列積	単位行列	$\times$
n 次元ベクトル空間の部分空間全体	和	零空間	$\bigcirc$

4.3. 競技プログラミングにおけるモノイド

n 個のデータを何らかの意味で結合して 1 個のデータとする，という処理は競技プログラミングにおいても頻出で，その多くはモノイドの言葉で定式化することができます。特に，

- 木において子の情報を集約する（木 dp）。
- 区間の情報を集約した値をセグメント木で扱う。

といった状況が，モノイドを使って定式化できることが多くあります（前者ではモノイドに可換性があることも多いでしょう）。

モノイドはセグメント木などのデータ構造で扱えることが有名で，4.2 節で挙げた例の大部分にはそのような用例を想定した問題が多数あります。また，同じ要素を n 個結合する x^n の計算に関する繰り返し二乗法は，モノイドでできるアルゴリズムです。

後述の群，環などと比べてもモノイドの例は多岐にわたり，4.2 節で挙げたものの以外のモノイドを見かけたり，問題に応じてモノイドを設計するような場面も多くあります。

5. 群

5.1. 定義

【定義 11】

モノイド $(G, *)$ は，任意の元が逆元を持つとき**群**（group）であるという。さらに $*$ が交換法則を満たすとき，**可換群**（commutative group）であるという。

群においては，加算に対する減算，乗算に対する除算のようなことが行えます。

【命題 12】

$(G, *)$ が群であるとき、任意の $a, b \in G$ に対して $a * x = b$ を満たす $x \in G$ がちょうどひとつ存在する。同様に $x * a = b$ を満たす $x \in G$ もちょうどひとつ存在する。

a^{-1} を a の逆元とします。 $a * x = b$ ならば、 $x = a^{-1} * a * x = a^{-1} * b$ となります。逆に $x = a^{-1} * b$ ならば $a * x = a * a^{-1} * b = b$ となるので、 $a * x = b \iff x = a^{-1} * b$ です。

同様に $x * a = b \iff x = b * a^{-1}$ であることも分かります。 ■

5.2. 具体例

S	二項演算	交換法則
$\mathbb{Z}$ (整数全体)	+	○
$\mathbb{Q}^\times$ (0 でない有理数全体)	$\times$	○
$\{0, 1, \dots, n-1\}$ ($\mathbb{Z}/n\mathbb{Z}$)	$(a+b) \bmod n$	○
素数 p に対して $\{1, 2, \dots, p-1\}$ (あるいは $\mathbb{F}_p^\times$)	$(ab) \bmod p$	○
$\mathbb{Z}_{\geq 0}$	XOR	○
全単射 $\mathbb{Z} \rightarrow \mathbb{Z}$ 全体	写像の合成	$\times$
$\{0, 1, \dots, n-1\}$ の順列全体	写像の合成	$\times$
$x \mapsto ax + b$ ($a, b \in \mathbb{F}_p, a \neq 0$) と書ける写像 $\mathbb{F}_p \rightarrow \mathbb{F}_p$ 全体	写像の合成	$\times$
$\det A \in \mathbb{F}_p^\times$ であるような $n \times n$ 行列 A 全体 ($\text{GL}(n, \mathbb{F}_p)$)	行列積	$\times$

5.3. 競技プログラミングにおける群

モノイドと比べると群はかなり構造が限定されて、競技プログラミングで頻出の例はそれほど多くありません。5.2 節で扱ったものあるいはその変種がほとんどだと思います。

n 個の要素に対する何らかの集約値を求める方法のうち、群である（逆元が存在する）ことを利用するものとしては、例えば累積和を用いた区間和クエリの処理が代表的です。また、Fenwick 木を用いた区間和の計算ではさらに可換群であることが前提となります。

また、Fermat の小定理は有限群（要素数が有限であるような群）にも一般化できます。証明は可換群の場合には [素数の性質](#) で扱っているものがそのまま適用できます。

6. 環

6.1. 定義

【定義 13】

集合 R に二項演算 $+$, $\times$ が定まっていて、以下の条件がすべて成り立つとき、組 $(R, +, \times)$ は**環** (ring) であるという。

1. $(R, +)$ は可換群である。
2. $(R, \times)$ は半群である。
3. **分配法則** (distributive law) が成り立つ、つまり任意の $a, b, c \in R$ に対して $a \times (b + c) = a \times b + a \times c$ および $(a + b) \times c = a \times c + b \times c$ が成り立つ。

$(R, +)$ の単位元は通常 0 , $(R, \times)$ の単位元は通常 1 と書く。 $(R, \times)$ の単位元が存在するとき、 $(R, +, \times)$ は**単位的** (unital) であるという。 $(R, \times)$ が交換法則を満たすとき、 $(R, +, \times)$ は**可換環** (commutative ring) であるという。

▶ 定義揺れについて

6.2. 具体例

$\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$

- 整数全体の集合を $\mathbb{Z}$,
- 有理数全体の集合を $\mathbb{Q}$,

- 実数全体の集合を $\mathbb{R}$,
- 複素数全体の集合を $\mathbb{C}$

と書きます。これらは通常の加法・乗法によって環をなします。

$\mathbb{Z}/n\mathbb{Z}$

集合 $\{0, 1, \dots, n-1\}$ は n を法とする加法・乗法によって環をなします ([合同式の基礎](#) 参照)。これは要素数が有限であるような環の例としても基礎的です。

なお $n=1$ とした場合 $\mathbb{Z}/1\mathbb{Z}$ は要素数が 1 つだけの環で、加法単位元 0 と乗法単位元 1 が一致します。この環は**零環** (zero ring) と呼ばれます。

ビット単位 AND, XOR

0 以上 $2^w - 1$ 以下の整数全体は、ビット単位の XOR を加法、AND を乗法として環をなします。この加法・乗法は 2 進法表記によって桁ごとに見れば、 $\mathbb{Z}/2\mathbb{Z}$ の加法・乗法であり、直積環と呼ばれる構成の例と考えることもできます。

多項式環・形式的べき級数環

R を環とすると、 R 係数の多項式全体 $R[x]$ は環をなします。この環を R 上の**多項式環** (polynomial ring) といいます。

同様に、 R 係数の形式的べき級数全体 $R[[x]]$ は環をなします。この環を R 上の**形式的べき級数環** (formal power series ring) といいます。

R をこれらの**係数環** (coefficient ring) といいます。

行列環

R を環とすると、 R の要素を成分とする $n \times n$ 行列全体 $M_n(R)$ は環をなします。これを R 上の**行列環** (matrix ring) といいます。

R が可換環である場合にも $M_n(R)$ は一般には可換環にならず、可換ではない環の代表例です。

6.3. 乗法モノイド，乗法群

【定義 14】

$(R, +, \times)$ を単位的環とすると、 $(R, \times)$ をその**乗法モノイド** (multiplicative monoid) という。

$\times$ について逆元を持つような R の要素全体を $R^\times$ とすると、 $(R^\times, \times)$ は群をなす。これを R の**乗法群** (multiplicative group) という。

例えば 5.2 節の具体例では、 $\mathbb{Q}^\times, \mathbb{F}_p^\times$ は乗法群となっています。また $\mathrm{GL}(n, \mathbb{F}_p)$ は $M_n(\mathbb{F}_p)$ の乗法群です。

6.4. モノイド代数 (畳み込み)

【定義 15】

R を環とし、 $(M, *)$ をモノイドとします。さらに、任意の $a \in M$ に対して $b * c = a$ を満たす組 (b, c) が有限個であるとし、

このとき、 M で添字付けられた R の元の組 $(f_a)_{a \in M}$ 全体は、次の演算によって環をなす。

- 加法を成分ごとの加法によって定める。つまり $(f_a)_{a \in M} + (g_a)_{a \in M} = (f_a + g_a)_{a \in M}$ と定める。
- 乗法 $(f_a)_{a \in M} \times (g_a)_{a \in M} = (h_a)_{a \in M}$ を、 $h_a = \sum_{b * c = a} (f_b \times g_c)$ によって定める。仮定よりこの和は有限和である。

この環を R 上の M の**モノイド代数** (monoid algebra) という。 $(f_a)_{a \in M}$ を、 $\sum_{a \in M} f_a x^a$ とも書く。

モノイド代数の乗法は**畳み込み** (convolution) ともいう。

環をなすことの証明（特に、分配法則および乗法の結合法則の証明）は省略します。

例えば $(M, *) = (\mathbb{Z}_{\geq 0}, +)$ とした場合が、形式的べき級数環です。

6.5. 半環

環の定義から加法逆元の存在を除いたような代数構造である**半環** (semiring) もしばしば登場します.

【定義 16】

集合 R に二項演算 $+$, $\times$ が定まっていて、以下の条件がすべて成り立つとき、組 $(R, +, \times)$ は**半環** (semiring) であるという.

1. $(R, +)$ は可換モノイドである.
2. $(R, \times)$ は半群である.
3. **分配法則** (distributive law) が成り立つ、つまり任意の $a, b, c \in R$ に対して $a \times (b + c) = a \times b + a \times c$ および $(a + b) \times c = a \times c + b \times c$ が成り立つ.
4. $(R, +)$ に関する単位元 0 について、次が成り立つ: 任意の $x \in R$ に対して $0 \times x = x \times 0 = 0$.

▶ 4 番目の条件について

特に次の例が重要です.

- $(\mathbb{Z} \cup \{\infty\}, \min, +)$ は半環をなす. 同様に $(\mathbb{Z} \cup \{-\infty\}, \max, +)$ は半環をなす.

このような半環を、**min-plus 半環** (min-plus semiring) や**トロピカル半環** (tropical semiring) などといいます.

半環上の行列半環を考えることもできます.

6.6. 競技プログラミングにおける環

競技プログラミングで現れる環として重要なものの大部分は上で取り上げたと思います. これらの環はいずれも、競技プログラミングで頻繁に現れる重要なものばかりです. ただし通常は、問題を解く際に、環の考え方が現れていると意識する必要はありません.

AtCoder Algorithm Lectures において「 R を環とする」といった説明が出てきた際には、まずは $\mathbb{Z}, \mathbb{R}, \mathbb{Z}/n\mathbb{Z}$ などを想定するようにしてください. 多項式環・形式的べき級数環・行

列環などを主に考える場合には、そのような記述があるはずですが、

多項式環や形式的べき級数環をはじめとして、様々なモノイド代数の畳み込み、行列環の乗法が、dp や数え上げなどの様々な場面で現れます。モノイド代数や行列環の乗法モノイドがモノイドである（結合法則を満たす）ことから、これらに繰り返し二乗法を用いたり、セグメント木で区間クエリを処理できるということも重要です。min-plus 半環上の行列も頻出です。

7. 体

7.1. 定義

【定義 17】

単位的可換環 $(F, +, \times)$ が、 $F^\times = F \setminus \{0\}$ を満たすとき、 $(F, +, \times)$ は**体** (field) であるという。

体においては、0 でない要素による除算が可能であり、通常の意味での「四則演算が行える代数構造」と考えることもできます。

7.2. 具体例

- $\mathbb{Q}, \mathbb{R}, \mathbb{C}$ は体です。
- $\mathbb{Z}/p\mathbb{Z}$ は、 p が素数のとき、そのときに限り体です。 p が素数のときこの体を $\mathbb{F}_p$ と書くのでした。
- 細かいことですが、零環あるいは $F = \mathbb{Z}/1\mathbb{Z}$ は体ではないものとして定義されています。 $F^\times = \{0\}$, $F \setminus \{0\}$ は空集合です。

$\mathbb{F}_p$ は要素数が有限であるような体です。このような体を**有限体** (finite field) といいます。一般に $q = p^f$ を素数のべき乗とすると、要素数が q であるような体が存在する（ある種の一意性も成り立つ）ことが知られています。

他に競技プログラミングで現れる体としては、体上の**有理式全体**や**形式的ローラン級数全体**がありますが、いずれも重要度はあまり高くありません。

7.3. 競技プログラミングにおける体

競技プログラミングにおける体の例としては、 $\mathbb{F}_p$ が圧倒的に重要です。巨大な数を n で割った余りを求めるというタイプの問題では実質的に環 $\mathbb{Z}/n\mathbb{Z}$ における計算を問われています。その中でも $n = p$ が素数の場合である $\mathbb{F}_p$ では、除算を含む計算の取り扱いがしやすくなるため、頻繁に出題されています。

一部の高度な問題では、 $\mathbb{F}_p$ 以外の有限体を利用するアルゴリズムも、登場することがあります。また、有限体の例として 2^{2^n} 未満の非負整数全体に適切に演算を定めた有限体である **nimber** も有名です。

8. 関連問題

特にありません。

本講座で扱った考え方は競技プログラミングの多くの場面で現れますし、関連する出題も多岐に渡りますが、そのような事例は AtCoder Algorithm Lectures 内の他の講座において言及します。

9. まとめ

本講座では、競技プログラミング（あるいは AtCoder Algorithm Lectures）での使用頻度が高い、代数構造に関する用語および、その代表例について解説しました。

これらの内容は、アルゴリズムやデータ構造が適用できる範囲を考えたり、この演算とこの演算の違いや共通点は何であるかといったことを考えて、理解を整理するのに役立つと思います。また、これまで別々の手法や定理だと理解していたものが、共通のものとして理解できるようになることもあると思います。

10. 参考文献

1. Wikipedia. 代数的構造. <https://ja.wikipedia.org/wiki/代数的構造>. (閲覧日: 2026-03-13).
2. Wikipedia. モノイド. <https://ja.wikipedia.org/wiki/モノイド>. (閲覧日: 2026-03-13).

3. Wikipedia. 群 (数学). [https://ja.wikipedia.org/wiki/群_\(数学\)](https://ja.wikipedia.org/wiki/群_(数学)). (閲覧日: 2026-03-13).
4. Wikipedia. 環 (数学). [https://ja.wikipedia.org/wiki/環_\(数学\)](https://ja.wikipedia.org/wiki/環_(数学)). (閲覧日: 2026-03-13).
5. Wikipedia. 半環. <https://ja.wikipedia.org/wiki/半環>. (閲覧日: 2026-03-13).
6. Wikipedia. 体 (数学). [https://ja.wikipedia.org/wiki/体_\(数学\)](https://ja.wikipedia.org/wiki/体_(数学)). (閲覧日: 2026-03-13).
7. Wikipedia. 有限体. <https://ja.wikipedia.org/wiki/有限体>. (閲覧日: 2026-03-13).
8. maspy. [多項式・形式的べき級数] (補足) 定義や式変形の正当性の確認.
[https://maspy.py.com/多項式・形式的べき級数-\(補足\)定義や式変形の正当性の確認](https://maspy.py.com/多項式・形式的べき級数-(補足)定義や式変形の正当性の確認). (閲覧日: 2026-03-13).
9. Kimiyuki Onaka, Shiho Midorikawa. セグメント木と代数構造の理論.
<https://elliptic-shiho.github.io/segtree/segtree.pdf>. (閲覧日: 2026-03-13).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)